

Modelagem MongoDB – Inserção de Dados e Consultas

FATEC Jacareí · Dia 3: Inserção de Dados e Consultas · 18/05/26

1. Inserção de Dados (seed.js)

O arquivo **seed.js** popula o banco **leads_db** com dados de teste. São criados IDs via ObjectId para manter as referências entre coleções.

IDs gerados para referenciar:

```
const users = Array.from({ length: 5 }, () => ObjectId())
const stores = Array.from({ length: 3 }, () => ObjectId())
const customers = Array.from({ length: 5 }, () => ObjectId())
const leads = Array.from({ length: 10 }, () => ObjectId())
```

users (5 documentos):

```
db.users.insertMany([
  { _id: users[0], name: "Admin", role: "administrador", active: true },
  { _id: users[1], name: "Gerente", role: "gerente", active: true },
  { _id: users[2], name: "Ana", role: "atendente", active: true },
  { _id: users[3], name: "Bruno", role: "atendente", active: true },
  { _id: users[4], name: "Carlos", role: "atendente", active: true }
])
```

stores (3 documentos):

```
db.stores.insertMany([
  { _id: stores[0], name: "Loja Centro", city: "São Paulo", state: "SP" },
  { _id: stores[1], name: "Loja Norte", city: "Guarulhos", state: "SP" },
  { _id: stores[2], name: "Loja Sul", city: "Santo André", state: "SP" }
])
```

customers (5 documentos):

```
db.customers.insertMany([
  { _id: customers[0], name: "João", phone: "1111", createdBy: users[2] },
  { _id: customers[1], name: "Maria", phone: "2222", createdBy: users[3] },
  { _id: customers[2], name: "Pedro", phone: "3333", createdBy: users[4] },
  { _id: customers[3], name: "Lucas", phone: "4444", createdBy: users[2] },
  { _id: customers[4], name: "Ana Paula", phone: "5555", createdBy: users[3] }
])
```

leads (10 documentos — exemplo dos 3 primeiros):

```
db.leads.insertMany([
  { _id: leads[0], customerId: customers[0], storeId: stores[0],
    attendantId: users[2], channel: "whatsapp", createdAt: new Date() },
  { _id: leads[1], customerId: customers[1], storeId: stores[1],
    attendantId: users[3], channel: "instagram", createdAt: new Date() },
  { _id: leads[2], customerId: customers[2], storeId: stores[2],
    attendantId: users[4], channel: "telefone", createdAt: new Date() },
  // ... 7 documentos adicionais
])
```

```
])
```

negotiations (10 documentos — exemplo dos 2 primeiros):

```
db.negotiations.insertMany([
  {
    leadId: leads[0], importance: "quente", status: "aberta",
    currentStage: "contato_inicial",
    history: [{ stage: "contato_inicial", status: "aberta",
    changedAt: new Date(), changedBy: users[2] }]
  },
  {
    leadId: leads[1], importance: "quente", status: "encerrada",
    currentStage: "fechamento",
    history: [
      { stage: "contato_inicial", status: "aberta", changedAt: new Date(),
      changedBy: users[3] },
      { stage: "fechamento", status: "encerrada", changedAt: new Date(),
      changedBy: users[3] }
    ]
  },
  // ... 8 documentos adicionais
])
```

logs (10 documentos):

```
db.logs.insertMany([
  { userId: users[0], action: "login", entity: "User", createdAt: new Date()
  },
  { userId: users[1], action: "create", entity: "Lead", createdAt: new Date()
  },
  { userId: users[2], action: "update", entity: "Customer", createdAt: new
  Date() },
  // ... 7 documentos adicionais
])
```

2. Consultas (consultas.js)

Sete consultas operacionais implementadas:

1. \$and — canal e atendente

Leads do canal whatsapp atendidos por Ana (users[2]).

```
db.leads.find({ $and: [ { channel: "whatsapp" }, { attendantId: users[2] } ] })
```

2. \$or — múltiplos canais

Leads de instagram ou whatsapp.

```
db.leads.find({ $or: [ { channel: "instagram" }, { channel: "whatsapp" } ] })
```

3. \$gt / \$lt — intervalo de datas

Leads criados em um intervalo de datas.

```
db.leads.find({ createdAt: { $gt: new Date("2020-01-01"), $lt: new Date("2030-01-01") } })
```

4. \$exists — campo presente

Negociações que possuem o array history.

```
db.negotiations.find({ history: { $exists: true } })
```

5. Projeção — campos selecionados

Retorna apenas nome e telefone dos clientes.

```
db.customers.find({}, { name: 1, phone: 1, _id: 0 })
```

6. sort — ordenação

Leads mais recentes primeiro.

```
db.leads.find().sort({ createdAt: -1 })
```

7. skip/limit — paginação

Página 2, com 5 registros por página.

```
db.leads.find().skip(5).limit(5)
```

3. Aggregations (aggregations.ts)

Cinco pipelines para indicadores gerenciais:

1. Leads por canal (channel)

Conta quantos leads vieram de cada canal.

```
db.leads.aggregate([
  { $group: { _id: "$channel", total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { origem: "$_id", total: 1, _id: 0 } }
])
```

2. Leads por status (via negotiations)

Distribuição das negociações por status.

```
db.negotiations.aggregate([
  { $group: { _id: "$status", total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { status: "$_id", total: 1, _id: 0 } }
])
```

3. Taxa de conversão

Percentual de negociações encerradas (convertidas).

```
db.negotiations.aggregate([
  { $group: { _id: null, total: { $sum: 1 },
    encerradas: { $sum: { $cond: [{ $eq: ["$status","encerrada"] },1,0] } } } },
  { $project: { total: 1, encerradas: 1,
    taxaConversao: { $multiply: [{ $divide: ["$encerradas","$total"] }, 100] } } }
])
```

4. Leads por atendente (\$lookup)

Quantidade de leads por atendente, com nome via \$lookup.

```
db.leads.aggregate([
  { $group: { _id: "$attendantId", total: { $sum: 1 } } },
  { $lookup: { from: "users", localField: "_id",
    foreignField: "_id", as: "user" } },
  { $unwind: "$user" },
  { $sort: { total: -1 } },
  { $project: { atendente: "$user.name", total: 1, _id: 0 } }
])
```

5. Leads por importância

Distribuição das negociações por importance (quente/morno/frio).

```
db.negotiations.aggregate([
```

```
{ $group: { _id: "$importance", total: { $sum: 1 } } },
{ $sort: { total: -1 } },
{ $project: { importancia: "$_id", total: 1, _id: 0 } }
])
```